Robust Proximal Adversarial Reinforcement Learning Under Model Mismatch

Peng Zhai, Xiaoyi Wei, Taixian Hou, *Member, IEEE*, Xiaopeng Ji, Zhiyan Dong, *Member, IEEE*, Jiafu Yi, and Lihua Zhang, *Member, IEEE*

***Abstract*—Reinforcement learning (RL) can generate high-performance control policies for complex tasks in simulation through an end-to-end approach. However, the RL policy is not robust to uncertainties caused by modeling mismatch between simulation and real environments, making it difficult to transfer to the real world. In response to the above challenge, this letter introduces a lightweight and efficient robust RL algorithm. The algorithm transforms the optimization objective of the adversary from a long-term cumulative reward to a short-term reward, making the adversary focus on the performance in the near future. Additionally, the adversarial actions are projected onto a finite subset within the perturbation space using projected gradient descent, effectively constraining the adversary's strength and obtaining more robust policies. Extensive experiments in both simulated and real environments show that our algorithm improves the generalization ability of the policy for the modeling mismatch, outperforming the next best prior methods across almost all environments.**

***Index Terms*—Reinforcement learning (RL), machine learning for robot control, robust/adaptive control.**

I. Introduction

DEEP reinforcement learning (DRL) has enabled agents to independently explore the optimal action through trial-and-error interaction with the environment, achieving end-to-end optimization from sensor to actuator. This technology has attracted more and more attention and has demonstrated remarkable achievements in robot control, autonomous driving, aerospace, and other fields [1]. However, training or deploying RL policies in the real world with high risk and complexity remains a significant challenge [2]. This is because data-driven RL algorithms may be expensive to train directly in the real environment: real-world data collection will be time-consuming, while unstable policies during the training phase may result in unsafe actuator actions or even catastrophic failures [3]. In contrast, training policies in a simulation environment and then deploying them in the real world is a cheaper and more promising approach [4].

Unfortunately, there is often a mismatch between the simulated and the real environments, such as unknown friction or load mass changes. These gaps can lead to significant degradation in policy performance and restrict the application of RL in fields with high requirements for reliability and safety, leading researchers to discuss and emphasize the robustness of RL [5].

Improving the robustness of RL policies is an effective deployment method in the real environment. The common methods in the field of robotics include domain randomization (DR) and adversarial reinforcement learning (ARL) [6]. The core idea of DR is to transform fixed distributions of physical parameters in the simulated environment into random distributions, aiming at covering the uncertainty distribution of corresponding parameters in the real environment [7]. Nevertheless, DR needs to integrate the expert experience to specify dynamically changing parameters, and the range of changes requires a lot of time to adjust carefully.

In the ARL architecture, the environment's approximate errors and parameter changes are modeled as perturbations generated by adversaries. The normal agent needs to maintain robustness against the adversary's perturbations while completing the original task goals. However, the adversary's policy is usually trained with RL algorithms. This random-exploration-driven policy optimization method makes it difficult to control the adversary's perturbations strength; neither too strong nor too weak of the adversary can effectively improve the robustness of the normal agent [8]. Furthermore, the training time for optimizing the adversary's policy will be non-negligible for complex tasks requiring extensive training.

In this letter, we propose a gradient-attacks-based robust proximal adversarial reinforcement learning (RPARL) method, which reduces the complexity and computational cost of adversarial training while improving the robustness of the normal agent's policy. The contributions of this work are: (1) We propose an asymmetric game adversarial framework that the adversary utilizes the n-step cumulative reward as the objective function, making it focus on the returns in the near future to generate

Received 27 April 2024; accepted 16 September 2024. Date of publication 3 October 2024; date of current version 9 October 2024. This article was recommended for publication by Associate Editor Jemin Hwangbo and Editor Jens Kober upon evaluation of the reviewers' comments. This work was supported in part by China Postdoctoral Science Foundation under Grant BX20220071 and Grant 2022M720769, and in part by the National Key R&D Program of China under Grant 2021ZD0113502 and Grant 2021ZD0113503. *(Corresponding authors: Jiafu Yi; Lihua Zhang.)*

Peng Zhai, Xiaoyi Wei, and Taixian Hou are with the Academy for Engineering and Technology, Fudan University, Shanghai 200433, China (e-mail: pzhai@fudan.edu.cn; weixy23@m.fudan.edu.cn; txhou21@m.fudan.edu.cn).

Xiaopeng Ji is with the State Key Lab of CAD&CG, Zhejiang University, Hangzhou 310058, China (e-mail: xp.ji@cad.zju.edu.cn).

Zhiyan Dong is with the Ji Hua Laboratory, Foshan 528251, China (e-mail: dongzhiyan@fudan.edu.cn).

Jiafu Yi is with the School of Information and Communication Engineering, Hainan University, Hainan 570228, China (e-mail: yjf1201@163.com).

Lihua Zhang is with the Engineering Research Center of AI and Robotics, Shanghai 200433, China (e-mail: lihuazhang@fudan.edu.cn).

This letter has supplementary downloadable material available at https://doi.org/10.1109/LRA.2024.3472348, provided by the authors.

Digital Object Identifier 10.1109/LRA.2024.3472348

2377-3766 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:50:17 UTC from IEEE Xplore. Restrictions apply.

adversarial perturbations, reducing the cumulative error due to perturbations in the long-term reward estimation. (2) We design a novel lightweight adversarial action generation method to directly generate perturbations through projected gradient descent (PGD) without policy optimization. This algorithm avoids the stochastic exploration of the adversary's policy and reduces the computational resources to optimize the adversary. (3) We extensively evaluate the proposed algorithm in various environments, and the results demonstrate the robustness of our method to environmental parameter variations. It is worth mentioning that we have transferred RPARL to the Legged_Gym, a massively parallel training environment for legged robots based on Isaac Gym. To our knowledge, this is the first robust ARL algorithm ported to Isaac Gym. Additionally, sim2real transfer tasks on the real quadrotor and legged robot are conducted to confirm the application potential of the RPARL in the real world.

II. Related Work

A. Adversarial Training in Reinforcement Learning

Adversarial training is one of the most popular and empirically successful techniques for enhancing deep neural networks (DNN) [9]. Morimoto et al. introduced adversarial training into the Markov Decision Processes (MDP) for the first time to strengthen the robustness of the policy [10]. Pinto et al. proposed the robust adversarial reinforcement learning (RARL) algorithm that utilizes the adversary to inject perturbation force into the environment [11]. Tessler proposed two adversarial RL algorithms [12]: the noisy action robust MDP (NR-MDP) adds continuous noise to the action to model the changes in physical parameters; the probabilistic action robust MDP (PR-MDP) simulates rare events by randomly replacing the normal agent's action. Huang et al. proposed the RRL-stack formulation, which treats ARL as a general-sum Stackelberg game while considering the sequentiality of the normal and adversarial agents during training [13]. Yi et al. combined ARL with safe RL to propose the adversarial constrained soft actor-critic algorithm (ACSAC) [14]. In the ACSAC, the adversary improves the robustness of the safety policy by altering noise parameters in the environment. However, the above methods require the adversary to be trained using RL algorithms, which are usually revealed as very complex and unstable.

B. Adversarial Attack in Reinforcement Learning

Adversary attacks are usually used to test and expose DRL's security vulnerabilities. Huang et al. extended the Fast Gradient Sign Method (FGSM) in deep learning to RL and reduced the trained RL agents' performance by attacking the observation data [15]. Kos and Song compared the effects of full-state perturbation and random noise, and they used value functions to guide attackers when to inject perturbations [16]. Lee et al. proposed a Myopic Action Space (MAS) attack that uses the gradient of the policy evaluation to generate a white box attack that superimposes the perturbation on the action of the normal agent at each timestep [17]. Qu et al. proposed Minimalistic attacks that degrade the normal agent performance by interfering with only a tiny fraction of observations or attacking with limited time steps [18]. Some researchers have constructed adversarial training based on these attack methods. Pattanaik et al. improved the robustness of the policy by perturbing observations with FGSM attacks during training [19]. Tan et al. used MAS for adversarial training to defend against similar attacks [20]. Lucas et al. dynamically adjusted training task complexity using adversarial attacks based on the critic network [21]. Different from the methods discussed in Section II-A, the adversarial training involved in this section does not require additional training of the adversary but requires knowledge of the internal components of the normal agent, such as the weight coefficients of the value network or the action network. In contrast, our approach requires neither training of the adversary nor additional information from the normal agent and only captures the states, actions, and rewards during the interaction.

III. Background

The ARL can model as a two-player zero-sum Markov game defined by the tuple $(S, A_1, A_2, P, R, \gamma, \rho)$, where S is the state set, A_1 and A_2 are the action sets of the normal and the adversarial agents, $P : S \times A_1 \times A_2 \times S \to \mathbb{R}$ is the state transition probability function, $R : S \times A_1 \times A_2 \to \mathbb{R}$ is the reward signal for both agents, γ is the discount factor, and ρ is the initial state distribution. Let π_μ denotes the normal agent's policy, and π_ν denotes the adversarial agent's policy. The reward function can be expressed as $R = E_{a\sim\pi_\mu,\omega\sim\pi_\nu}(r(s, a, \omega))$, where $a \sim \pi_\mu(\cdot \mid s) \in A_1$ and $\omega \sim \pi_\nu(\cdot \mid s) \in A_2$. The ARL assumes that if the normal agent can learn a policy that is robust to all perturbations from adversarial agent, then this policy will be robust to model mismatch in deployment or testing scenarios. The goal of the normal agent is to maximize the return (i.e., the cumulative reward function),

$$G_{\pi_\mu} = E_{s_0\sim\rho,a\sim\pi_\mu,\omega\sim\pi_\nu}\left[\sum_{t=0}^{T-1}\gamma^t r(s_t, a_t, \omega_t)\right], \quad (1)$$

where T is the horizon length of each episode. The adversarial agent is used to generate perturbation actions, which can be defined as different types of controllers. For example, in RARL and NR-MDP, the adversary's policy is parameterized by a neural network with the same structure as the normal agent. The adversary's objective is also to maximize its return, but the difference is that the adversary's return is opposite to that of the normal agent:

$$G_{\pi_\nu} = E_{s_0\sim\rho,a\sim\pi_\mu,\omega\sim\pi_\nu}\left[\sum_{t=0}^{T-1}-\gamma^t r(s_t, a_t, \omega_t)\right]. \quad (2)$$

It can be seen that maximizing the adversary's return is equivalent to minimizing the normal agent's return. The normal and the adversarial agents constitute a zero-sum game. Let $G(\pi_\mu, \pi_\nu) = G_{\pi_\mu} = -G_{\pi_\nu}$ denote the hybrid return of the game, Perolat et al. proved that if the optimal hybrid return (equilibrium return) G^* exists, it is equivalent to the minimax equilibrium and Nash

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:50:17 UTC from IEEE Xplore. Restrictions apply.

equilibrium [21]:

$$G^* = \min_{\pi_\nu} \max_{\pi_\mu} G(\pi_\mu, \pi_\nu) = \max_{\pi_\mu} \min_{\pi_\nu} G(\pi_\mu, \pi_\nu), \quad (3)$$

Pinto et al. pointed out that the optimal return of the Nash equilibrium can be approximated by alternating policy iteration between the normal and the adversarial agents, thereby avoiding the high computational resources to compute the equilibrium solution at each iteration [11]. Depending on the task and environment, the adversary's scope can encompass various components of RL, such as state space, action space, or transition function [22]. Adversarial perturbations can be introduced to the state space, which can model sensor errors or faults, and to the action space or transition function, which can model sudden changes in load, friction, or external perturbations. Compared to the transition function, setting the action space to the adversary's scope does not require any additional environmental modification, which is more simple and convenient to implement [23]. The actions of the normal and the adversarial agents are weighted to generate a hybrid action $\hat{a}$ of the input environment,

$$\hat{a} = (1-\alpha)a + \alpha\omega, \quad (4)$$

where $\alpha \in [0, 1]$ is the weighting coefficient that determines the strength of the adversary during training phase. When α is set to 0, the adversary cannot impact the environment, and the game model degenerates to the standard MDP.

IV. Methodology

In this section, we introduce a PGD-based ARL algorithm, RPARL, to address the challenge of robustness to various environmental parameter changes and perturbations in RL algorithms while reducing the computational cost of adversarial training. This algorithm defines the adversary's scope as the action space to achieve a more convenient environmental transition ability. The difference is that we do not strictly adhere to a zero-sum game formulation to optimize the adversarial players. In other words, the normal and adversarial agents do not maximize and minimize the same objective function. In this case, the optimization objective of the normal agent is still to maximize the cumulative rewards under the adversary's perturbation,

$$\max_{\theta\in\Theta} E_{s_0\sim\rho, \hat{a}\sim\pi^{mix}} \left[\sum_{t=0}^{T-1} \gamma^t r(s_t, \hat{a}_t) \right], \quad (5)$$

where $\hat{a}$ is the hybrid action, generated by the joint-policy π^{mix}:

$$\forall s \in S, \hat{a} \in \hat{A}, \pi^{mix}(\hat{a} \mid s) \equiv E_{a\sim\pi_\mu, \omega\sim\pi_\nu} \left[\mathbb{1}_{\hat{a}=a+\omega} \right], \quad (6)$$

where $\mathbb{1}$ is indicator function, the normal agent's policy π_μ is parameterized by $\theta \in \Theta$, and $\hat{A}$ is the hybrid action space. In contrast to (4), our method does not use the weighting coefficient α of the hybrid action, which determines the adversarial strength. In the subsequent sections, we will discuss the constraint on the adversary's strength during the PGD optimization process. For the adversary's policy π_ν, we let the adversary only focus on the short-term rather than the cumulative rewards of the whole episode to generate perturbations. This approach helps reduce the error caused by perturbations when estimating the cumulative reward. To this end, we define the adversary's optimization objective as:

$$\min_{\omega_t} G_p(s_t, \hat{a}_t) = \sum_{j=t}^{t+H} r(s_j, \hat{a}_j)$$
$$\text{subject to : } \|\omega_t\|_L \le \delta,\ \hat{a}_t = a_t + \omega_t, \quad (7)$$

we refer to $G_p(s_t, \hat{a}_t)$ as the proximal return, which represents the cumulative sum of rewards for the recent H time-steps. $\|\cdot\|_L$ denotes the L-norm of the adversarial perturbations, $\delta \ge 0$ denotes the strength constraint of each adversarial perturbation, which has different physical meanings according to different norms. For example, L_2 norm denotes the constraint on the energy of the perturbation, and L_∞ norm denote the constraint on the maximum magnitude of the perturbation. In the RPARL, we employ the L_2 norm. When $\delta = 0$, our method degenerates to standard RL. To avoid dimensionality catastrophe, the proximal return is calculated using a neural network $G_{\theta_p}(s, \hat{a})$ for fitting through the data-driven method. The loss function of $G_{\theta_p}(s, \hat{a})$ is defined as:

$$\mathcal{L}_{\theta_p} = E_{(s,\hat{a})\sim D_p} \left[\frac{1}{2} (G_{\theta_p}(s, \hat{a}) - G_p(s_t, \hat{a}_t))^2 \right], \quad (8)$$

where θ_p represents the parameters of the proximal return network, D_p denotes the rollout buffer (i.e. the trajectory data of the interaction process), which stores the tuples of states, hybrid actions, and rewards during training. The adversary aims to minimize the proximal return, which represents a nonlinearly constrained optimization problem and can be solved using the PGD method. Thus, we generate the adversary's action ω_t using (7). Specifically, at each time-step, we sample a random action $a'_t \in A_1$ from the normal agent's action space as the initial value for the adversarial action. Taking $\hat{a}'_0 = a'_t$ as the initial hybrid action of stochastic optimization, the gradient descent algorithm is used to obtain one-step optimization of the unconstrained hybrid action $\hat{a}'_t$:

$$\hat{a}'_t = a'_t - \xi \nabla_{\hat{a}'_0} G_{\theta_p}, \quad (9)$$

where $\nabla_{\hat{a}'} G_{\theta_p}$ represents the gradient of the proximal return w.r.t. the hybrid action and ξ is optimization step size. Based on our definition of hybrid actions, it can easily obtain the unconstrained adversarial perturbation ω'_t by subtracting the normal agent's action a_t from $\hat{a}'_t$. Finally, the adversarial action ω_t is obtained by projecting ω'_t onto a ball with radius δ in the adversarial action space:

$$\omega'_t = \hat{a}'_t - a_t$$
$$\omega_t = \begin{cases} \omega'_t, & \|\omega'_t\|_2 \leqslant \delta \\ \delta\omega'_t / \|\omega'_t\|_2, & \|\omega'_t\|_2 > \delta. \end{cases} \quad (10)$$

We summarize RPARL in Algorithm 1, where N_{iter} is the total number of rollouts, N_{Sample} is the length of each rollout trajectory, and D_μ is the normal agent's rollout buffer. In implementing the PGD algorithm, we utilize multi-step gradient descent to solve the unconstrained hybrid action. Additionally, we set convergence criteria ε and the maximum number of

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:50:17 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: RPARL.

Input Adversarial strength constraint δ, proximal return horizon H, PGD step size ξ, PGD convergence criteria ε.
Initialize Rollout buffer D_μ, D_p.
Initialize Normal agent policy $\pi_\mu(a|s)$, proximal return network $G_{\theta_p}(s,\hat{a})$, and corresponding parameters θ, θ_p.
for k in N_{iter} **do**
  for t in N_{Sample} **do**
    Sample normal agent's action $a_t \sim \pi_\mu(a|s,\theta)$
    Sample initial adversarial action a'_t from A_1
    $i=0$, $\hat{a}'_i = a'_t$, $\hat{a}'_{i+1} = \hat{a}'_i - \xi \nabla_{\hat{a}'_i} G_{\theta_p}$
    while $\hat{a}'_{i+1} - \hat{a}'_i \geqslant \varepsilon$ and $i \leqslant N_{PGD}$ **do**
      $\hat{a}'_{i+1} = \hat{a}'_i - \xi \nabla_{\hat{a}'_i} G_{\theta_p}$, $i+=1$
    end while
    $\hat{a}'_t = \hat{a}'_{i+1}$, $\omega'_t = \hat{a}'_t - a_t$
    Calculate the adversarial action ω_t using (10)
    Execute hybrid action $\hat{a}_t = a_t + \omega_t$, observe reward r_t and new state s_{t+1}
    Store transitions (s_t, a_t, r_t, s_{t+1}) in D_μ, store $(s_t, \hat{a}_t, r_t)$ in D_p
  end for
  Update θ using PPO and rollout buffer D_μ
  Update θ_p using Adam and rollout buffer D_p
end for

iterations N_{PGD} for PGD to improve computational efficiency. The proximal return network $G_{\theta_p}(s,\hat{a})$ for PGD gradient computation can be updated by the popular Adam optimizer in deep learning, and the normal agent adopts the PPO algorithm to update policy [24]. This process is repeated until convergence.

It's worth noting that in RPARL, the adversarial agent's policy is not explicitly defined by the neural network, and the adversarial action is directly generated by the PGD of the proximal reward. Therefore, the adversary does not need to perform complex policy optimization throughout the training process, which makes RPARL a more efficient and lighter adversarial training method.

V. Experiments and Results

Our experiments aim to investigate: 1) Does our method improve robustness to model mismatch and external perturbations? 2) Can our method transfer efficiently in the sim2real task? To answer these questions, we designed six groups of experiments. The first four groups of experiments comprehensively analyzed the robustness of the proposed algorithm to different environmental parameters and external perturbations. The first group of experiments was performed in the MuJoCo environment to compare the different algorithms' robustness in the face of the mass change of the robot body in the robot manipulation task. The second group of experiments simulated the Lunar Lander task, where we compared the success rates of different algorithms under variations in gravity and mass. In the third group of experiments, we transplanted RPARL into the Legged_Gym, a legged robot training environment of the parallel simulation platform Isaac Gym. We compared the success rates of different algorithms under the legged robot's mass and friction changes. The fourth group of experiments was conducted in the GymFc, an RL environment for developing attitude flight controllers for the quadrotor. We evaluated the performance of different algorithms under actuator perturbations. In the last two groups of experiments, we deployed policies trained in simulation environments onto the real legged robot and real racing drone to demonstrate RPARL's potential for sim2real tasks. Note that the adversary is used to train the normal agent to acquire a robust policy and does not participate in the testing and deployment of the normal agent's policy.

A. Experimental Settings

1) MuJoCo Environment: We select four tasks from a modified adversarial Gym [11]: InvertedPendulumAdv1, InvertedDoublePendulumAdv-v1, HalfCheetahAdv-v1, and HopperAdv-v1 to test our algorithm. The reward functions for all environments can be found in [11]. We used the PPO algorithm implemented by OpenAI baselines as the policy optimizer for all the algorithms to compare the baseline algorithms to ours fairly. Specifically, we compared Vanilla PPO [24], RARL [11], NR-MDP [12], MAS [20], and DR. In RARL, the adversary perturbation is the external force acting on the robot body. In NR-MDP and MAS, the adversary perturbation is carried out by the noise superimposed on the action. We called the DR as the Oracle method because it is trained directly on the test domain.

2) LunarLander Environment: We adopt LunarLander-Continuous-v2 in OpenAI Gym to simulate the spacecraft landing task on an unknown planet [20]. The objective of the task is to land safely at randomly generated locations while minimizing the use of thrusters. Similar to the MuJoCo environment, we compared the landing success rate and cumulative reward of RPARL with five baseline algorithms.

3) Legged_gym Environment: We transplanted RPARL into the Legged_Gym environment of NVIDIA's Isaac Gym, a massively parallel simulator, allowing us to complete training within half an hour [25]. Based on this achievement, we further selected Solo8, an open-source quadruped robot that performs a wide range of physical actions, as the robot platform for testing and deploying our algorithm [26].

4) GymFc Environment: GymFc is a flight control tuning framework with a focus on attitude control developed by Willam [27]. GymFc's novel approach fuses together the digital twinning paradigm with flight control training to provide seamless transfer to hardware. In each episode, the GymFc console randomly generates three axial desired angular velocities, and the agent's task is to track them.

5) Real Environment: We compare the simulation and the real environment in Fig. 2. Deploying Solo8 in a real-world environment requires overcoming various impacts caused by model mismatches, such as actuator delay, encoder zero-position errors, and friction changes. The quadrotor in the real environment has a higher mass and asymmetrical frame, which makes it have a significant sim2real gap.

B. Robustness Under the Mass Change

In the MuJoCo environment, we compared the ability of control policies to adapt to changes in robot mass. Mass changes

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:50:17 UTC from IEEE Xplore. Restrictions apply.



Fig. 1. Flight trajectory of the RPARL controller while perform aerial maneuvers with large payloads.

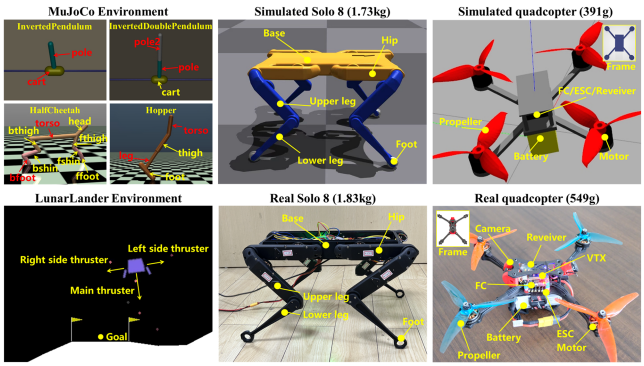


Fig. 2. Simulation and real environments we used to evaluate algorithms.

can not only simulate changes in the robot's load or joint abrasion but also affect the magnitude of friction, making it an effective way to test the robustness of algorithms. Specifically, the policy is trained with fixed robot mass in the training domain. In the test domain, we changed the masses of the two body parts of the robot (the changed body parts are marked by red font in Fig. 2). The range of mass change is [0.1, 2.1] with an interval of 0.2, and the two body parts are combined into 100 groups of different mass combinations. Seven random seeds initialize the trained policies, and 700 episodes are tested for each mass group. We define task success as the policy of consistently controlling the robot's motion for a certain number of steps without falling; otherwise, the task fails.

The results in Table I demonstrate that the RPARL algorithm outperforms all baselines in most environments. In the InvertedPendulum, the RPARL and Oracle algorithms have comparable adaptability, while in the InvertedDoublePendulum and Hopper environments, the RPARL algorithm demonstrates a higher success rate in a wider range of mass combinations, outperforming other baseline methods significantly. It can be noted that the MAS algorithm performs poorly in the InvertedDoublePendulum and Hopper environments. This might be attributed to MAS

TABLE I
SUCCESS RATES (%) AND STANDARD DEVIATIONS OF DIFFERENT ALGORITHMS WITH 100 MASS COMBINATIONS ARE COMPARED. THE PENDULUM AND DOUBLE DENOTE THE INVERTEDPENDULUM AND INVERTEDDOUBLEPENDULUM ENVIRONMENT, RESPECTIVELY. THE BEST RESULTS IN EACH CATEGORY ARE IN BOLD

Algorithm	Pendulum	Double	HalfCheetah	Hopper
Vanilla PPO	59.2±0.4	36.3±1.19	66.7±2.24	14.6±0.49
RARL	72.8±0.75	40.6±1.62	82.5±1.96	20.4±0.66
NR-MDP	71.2±0.4	32.5±1.67	84.6±1.91	28.2±0.98
Oracle	**78.0±0.2**	33.3±1.49	84.3±1.27	22.0±0.45
MAS	76.2±0.35	24.2±1.6	78.0±2.03	22.6±1.02
RPARL	**78.0±0.28**	**53.7±0.64**	**88.8±2.31**	**46.4±0.49**

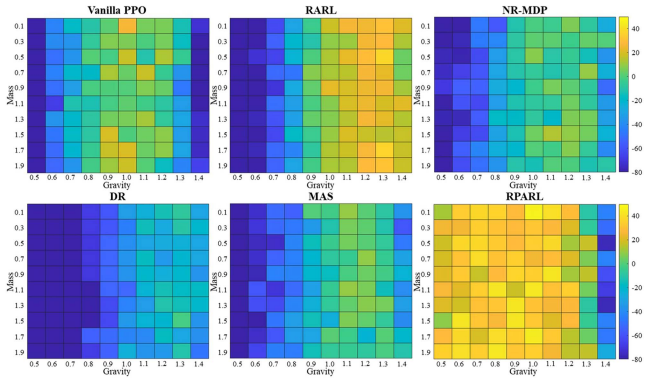


Fig. 3. Difference between the average return of each algorithm and the benchmark return. The x- and y-axes represent the spacecraft's mass change and the planet's gravity change, respectively.

being a white-box attack method, which focuses on generating effective perturbations [17], thereby increasing the complexity of training the normal agent's policy in the perturbation-sensitive tasks such as InvertedDoublePendulum and Hopper. In contrast, our approach emphasizes generating robust policies rather than formidable adversaries. See *video_1* for visual performance and training time of different algorithms.

C. Robustness Under the Mass and Gravity Change

In the LunarLander environment, we compared the robustness of the control policies to handle changes in spacecraft mass and planet gravity to simulate complex scenarios that might arise during planetary landing missions. Specifically, the policies were trained with fixed mass and gravity. In the test domain, we simultaneously change the mass and gravity. The mass change ranged from [0.1, 2.1] with an interval of 0.2, and the gravity change ranged from [0.5, 1.5] with an interval of 0.1. These two environmental parameters were combined into 100 groups with different combinations. Similar to Experiment B, we used seven random seeds to initialize the trained policies, and each group was tested with 700 episodes. We define task success as the lander landing safely at the target point and turning off the engines. If the lander crashes, the task fails.

We take the average return of the PPO algorithm in the training domain as the benchmark return and draw the heatmaps of each algorithm's returns subtracted from the benchmark return. Fig. 3 illustrates that the RPARL algorithm significantly outperforms

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:50:17 UTC from IEEE Xplore. Restrictions apply.

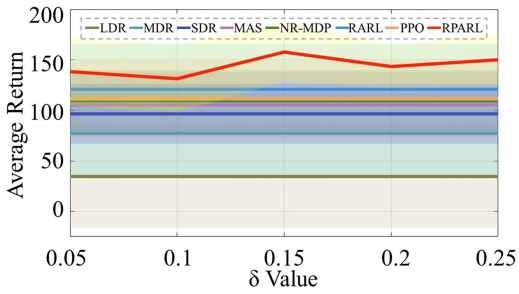


Fig. 4. Ablation study of δ in LunarLander environment. Algorithms besides RPARL are fixed comparison baselines, so performance does not vary with δ.

the benchmark return in most scenarios, even in low-gravity environments where other algorithms perform poorly. The *video_1* recorded the behavior of different agents, and it can be observed that RPARL can still land safely in a low gravity environment, while other algorithms usually cause the lander to crash (shut down the main engine before landing). Counter-intuitively, RPARL achieves higher returns than the benchmark return, even near the training domain (center of the heatmap). This implies that RPARL can improve the control performance and robustness of the policy at the same time. Similar findings were reported by [11] that the RARL algorithm improves the return value of the training domain through the introduction of perturbation. The possible reason is that appropriate adversarial perturbations can introduce beneficial randomness during training, forcing agents to experience more states, which indirectly improves exploration efficiency and thus enhances the performance of the policy.

1) Ablation Study of Adversarial Strength Constraint δ: We set the range of δ to [0.05, 0.25]. In addition, we consider 3 DR settings: SDR with mass range [0.9, 1.2] and gravity range [0.9, 1.1] (The setting of DR in Fig. 3), MDR with mass range [0.8, 1.4] and gravity range [0.8, 1.2], LDR with mass range [0.7, 1.4] and gravity range [0.7, 1.3]. The average cumulative return of all algorithms under 100 sets of test parameters is shown in Fig. 4. Although RPARL performance fluctuates with δ, it still outperforms other algorithms. It is worth noting that the DR algorithm with different random ranges does not perform well in the LunarLander environment, and further increasing the randomization range of mass or gravity will lead to a sharp decline in performance. This suggests that the DR method does not consistently achieve average robustness across all environmental distributions. The DR method may degrade policy performance in tasks sensitive to environmental parameters.

D. Robustness Under the Mass and Friction Change

In the Legged_Gym environment, we compared the robustness of control policies against variations in the Solo8 robot's base mass and ground friction. In the testing domain, we added extra base mass in the range of [0.5, 2.75] with 0.25 intervals and changed the ground friction coefficient within the same numerical range. These two environmental parameters were combined into 100 groups with different combinations. Task success was defined as the robot maintaining continuous movement for a prescribed number of steps without falling. In the training domain,

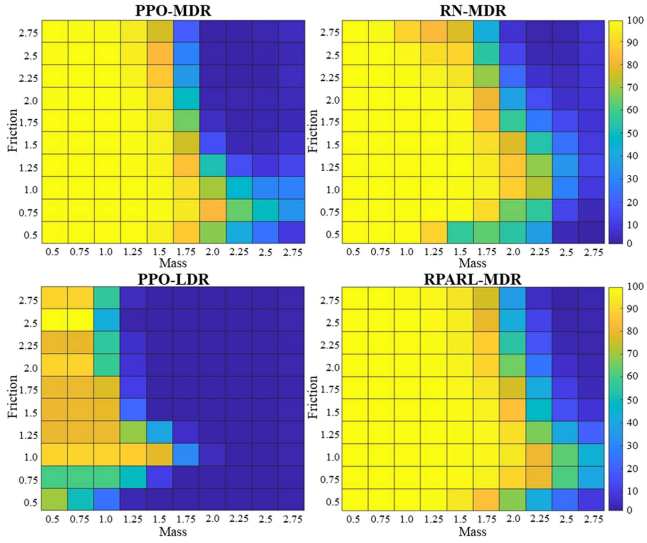


Fig. 5. Average success rates of each algorithm. The x- and y-axes represent the Solo8's extra base mass change and the ground friction coefficient change, respectively.

we established two DR settings: PPO-MDR with extra mass variations of $[-0.2, 1.5]$ and friction coefficient variations of [0.9, 1.8], and PPO-LDR with extra mass variations of [0.5, 2.25] and friction coefficient variations of [0.5, 2.25]. To assess the adaptability of our approach, we conducted joint training using PPO-MDR's DR setting alongside RPARL (RPARL-MDR). For comparison, we also trained PPO-MDR in combination with random action noise (RN-MDR), which is a common technique to improve policy robustness in the robotic environment.

The success rate heatmap in Fig. 5 shows that PPO-MDR has limited adaptability within the preset range of friction and mass. Once the mass exceeds the range set during training, the success rate will decline rapidly. RN-MDR marginally improves the success rate under heavier masses but reduces its success rate in the case of low friction. In contrast, RPARL can be combined with DR to effectively improve adaptability outside the training domain and obtain the best robustness improvement. In addition, simply increasing the randomization range in the training domain leads to the worst policy robustness (PPO-LDR). This could be attributed to the fact that a large DR range increases the complexity of policy learning and leads to training failures, which is also confirmed by the results in Experiment C.

E. Robustness Under the Action Noise

In the design of the flight controller, Fei et al. reported that the perturbation to the motor signal (i.e., action noise) might significantly degrade the flight controller's performance and cause severe failure [28]. Therefore, in the test domain of the GymFc environment, we superimpose 10% random noise on the normal agent's action, which makes the perturbation act directly on the motors of the quadrotor. Fig. 6 presents different algorithms' mean absolute errors (MAE) in tracking 700 groups of randomly generated target angular velocities. The results show that compared to vanilla PPO, the robust RL algorithms based on action perturbations can effectively reduce tracking errors

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:50:17 UTC from IEEE Xplore. Restrictions apply.

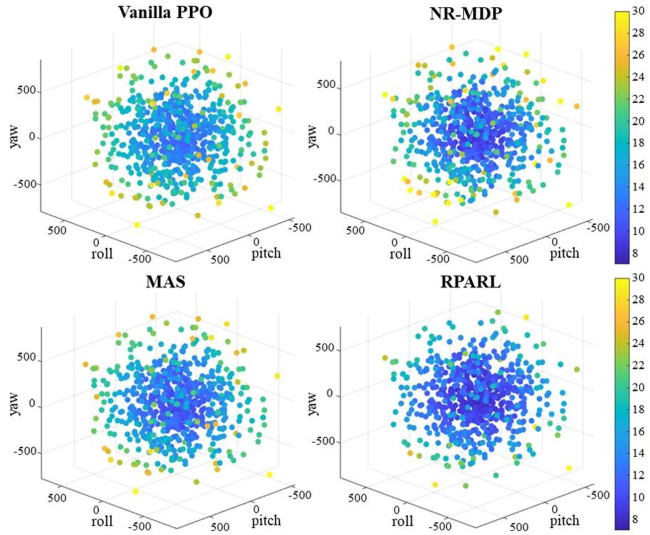


Fig. 6. Scatter plot of the GymFc showing tracking error for each target angular velocities, which are randomly sampled from Gaussian distributions with $\mu = 0$ and $\sigma = 500$.

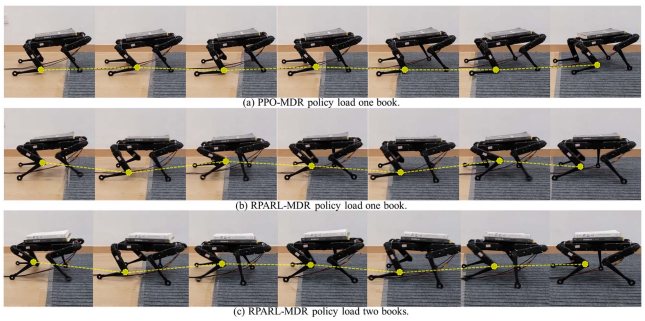


Fig. 7. Sim2Real experiments conducted on the real Solo8 robot: the policy needs to control Solo8 to cross the ground with different friction and loads. The *video_1* demonstrates these tests clearly.

under motor noise. Additionally, RPARL exhibits a smaller and more consistent error range. Even for target points with higher angular velocities, as shown in the periphery of Fig. 6, RPARL maintains a relatively small tracking error. This signifies that the proposed algorithm can adapt to more aggressive flight attitudes (i.e., flying with larger angular velocities), and the generated controllers have higher maneuverability.

F. Sim-to-Real Task

1) Real Solo8 Robot: We tested the robustness of PPO-MDR and RPARL-MDR policies on the real Solo8 robot. We simulated changes in load by placing books of different weights on the back of Solo8 and simulated grounds with different frictions through wooden floors and carpets. Fig. 7 illustrates the walking gaits corresponding to the different policies:7 a for the PPO-MDR, whereas Fig. 7(b) and (c) for the RPARL-MDR. The yellow dashed line marks the movement path of the robot's hind leg joints. It can be seen that despite base mass randomization in the simulation, the action output of the PPO-MDR still struggles to support the load (a single book weighing 0.72 kg), and the hind

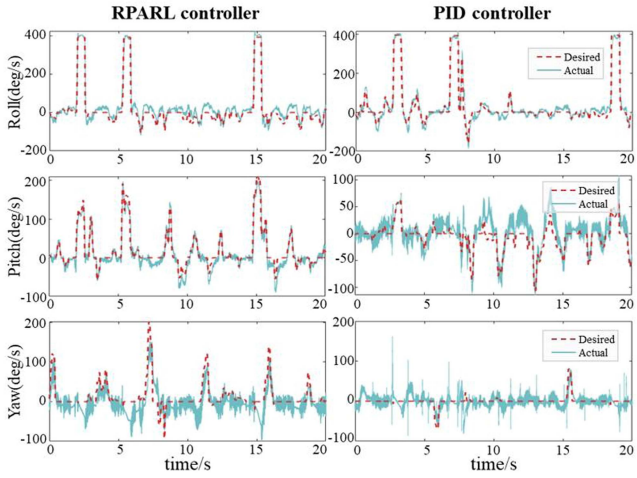


Fig. 8. Flight data of RPARL and PID-based flight controller. Red curve shows the remote-control input, whereas the cyan curve shows the angular velocity measured by the IMU. The flight controller sampling frequency is 0.01 s, whereas the sampling duration is 20 s.

TABLE II
NORMALIZED MAE OF PITCH, ROLL, AND YAW AXES OF RPARL AND PID-BASED ATTITUDE CONTROLLERS IN THE REAL WORLD

Controller	Roll (deg/s)	Pitch (deg/s)	Yaw (deg/s)
PID	**0.048**	0.125	0.123
RPARL	0.056	**0.046**	**0.102**

legs are restricted to a small range of movement. This gait results in lower walking efficiency on smooth floors. In contrast, the action output of the RPARL-MDR allows Solo8's legs to support the additional mass strongly. Even with a further increase in load mass (two books totaling 1.26 kg, which is 68.8% of the Solo8's weight), it can maintain a natural and efficient walking gait. More experimental comparisons and details can be found in *video_1*.

2) Real Racing Drone: We transferred the RPARL policy trained in GymFC to the Betaflight firmware, replacing the original PID controller for attitude tracking. The onboard black box records gyroscope and remote command data during the flight, which allows us to analyze the controller's tracking performance. Fig. 8 shows the tracking curves of RPARL and the PID controller for remote control commands, and Table II presents the normalized MAE for all three axes. The results show that the RPARL policy has comparable performance with the PID controllers in the roll axis, while in the pitch and yaw axes, the RPARL still maintains a smaller tracking error despite having a larger desired angular velocity. Additionally, it can be observed that all controllers have oscillations in the yaw axis. The possible reason is that the high maneuverability of the quadrotor relies on the drastic change of the rotors' angular velocity, which introduces additional torque noise in yaw control, leading to increased tracking errors.

Finally, we make the quadrotor perform maneuver actions of aerial rolls while carrying a large payload. For a small UAV, approximately 10–100 mW is required for 1 g additional take-off weight for hovering [29]. When executing aerial maneuvers, the controller needs to track larger angular velocities, which

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:50:17 UTC from IEEE Xplore. Restrictions apply.

consume more power. Even small additional loads can impact the aircraft's performance, which places higher demands on the controller's tracking capabilities. In our experiments, we make the quadrotor carry 98.68 g and 162.66 g loads, equivalent to 17.9% and 29.7% of the total weight of the quadrotor. The flight trajectory is shown in Fig. 1. The quadrotor performed the roll motion accurately and quickly while carrying a heavy load. This indicates that the RPARL algorithm can not only effectively bridge the gap between simulation and the real environment but also has good generalization ability in the case of significant parameter changes in the real environment.

VI. Conclusion

This letter proposes RPARL, a simple and efficient robust reinforcement learning algorithm. The critical idea of this algorithm is to adjust the adversary's objective to minimize the proximal return instead of the entire episode return so that the adversary becomes more focused on reducing the short-term rewards of the normal agent, avoiding biases in the estimation of long-term rewards caused by perturbations. Additionally, the RPARL employs the PGD algorithm to generate adversary actions without optimizing the adversary's policy and effectively reducing the computational resources and time for adversarial training. Experiments conducted on simulation and sim2real tasks prove that the RPARL algorithm outperforms other baseline methods in robustness. The test results in various environments further illustrate the algorithm's adaptability.

In the future, we will develop and refine our algorithms for more complex robotic tasks based on the parallel training environment Isaac Gym and provide utilities for rapid deployment from simulation to the real world. Furthermore, improving the robustness of multi-agent RL algorithms is also an important research direction.

References

[1] C. Li, P. Zheng, Y. Yin, B. Wang, and L. Wang, "Deep reinforcement learning in smart manufacturing: A review and prospects," *CIRP J. Manuf. Sci. Technol.*, vol. 40, pp. 75–101, 2023.

[2] B. Singh, R. Kumar, and V. P. Singh, "Reinforcement learning in robotic applications: A comprehensive survey," *Artif. Intell. Rev.*, vol. 55, no. 2, pp. 945–990, 2022.

[3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.

[4] W. Zhu, X. Guo, D. Owaki, K. Kutsuzawa, and M. Hayashibe, "A survey of sim-to-real transfer techniques applied to reinforcement learning for bioinspired robots," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 7, pp. 3444–3459, Jul. 2023.

[5] E. Salvato, G. Fenu, E. Medvet, and F. A. Pellegrino, "Crossing the reality gap: A survey on sim-to-real transferability of robot controllers in reinforcement learning," *IEEE Access*, vol. 9, pp. 153171–153187, 2021.

[6] H. Nguyen and H. La, "Review of deep reinforcement learning for robot manipulation," in *Proc. IEEE 3rd Int. Conf. Robotic Comput.*, 2019, pp. 590–595.

[7] R. Polvara, M. Patacchiola, M. Hanheide, and G. Neumann, "Sim-to-real quadrotor landing via sequential deep Q-networks and domain randomization," *Robotics*, vol. 9, no. 1, 2020, Art. no. 8.

[8] P. Zhai, T. Hou, X. Ji, Z. Dong, and L. Zhang, "Robust adaptive ensemble adversary reinforcement learning," *IEEE Robot. Automat. Lett.*, vol. 7, no. 4, pp. 12562–12568, Oct. 2022.

[9] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–10.

[10] J. Morimoto and K. Doya, "Robust reinforcement learning," *Neural Computation*, vol. 17, no. 2, pp. 335–359, 2005.

[11] L. Pinto, J. Davidson, R. Sukthankar, and A. Gupta, "Robust adversarial reinforcement learning," in *Proc. 34th Int. Conf. Mach. Learn.*, 2017, pp. 2817–2826.

[12] C. Tessler, Y. Efroni, and S. Mannor, "Action robust reinforcement learning and applications in continuous control," in *Proc. Mach. Learn. Res.*, Jun. 2019, vol. 97, pp. 6215–6224.

[13] P. Huang, M. Xu, F. Fang, and D. Zhao, "Robust reinforcement learning as a Stackelberg game via adaptively-regularized adversarial training," in *Proc. 31st Int. Joint Conf. Artif. Intell.* 2022, pp. 3099–3106.

[14] Z. Yi, Y. Xu, and C. Wu, "Model-free economic dispatch for virtual power plants: An adversarial safe reinforcement learning approach," *IEEE Trans. Power Syst.*, vol. 39, no. 2, pp. 3153–3168, Mar. 2024.

[15] S. Huang, N. Papernot, I. Goodfellow, Y. Duan, and P. Abbeel, "Adversarial attacks on neural network policies," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–4.

[16] J. Kos and D. Song, "Delving into adversarial attacks on deep policies," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–4.

[17] X. Y. Lee, S. Ghadai, K. L. Tan, C. Hegde, and S. Sarkar, "Spatiotemporally constrained action space attacks on deep reinforcement learning agents," in *Proc. AAAI Conf. Artif. Intell.*, Apr. 2020, vol. 34, no. 04, pp. 4577–4584.

[18] X. Qu, Z. Sun, Y.-S. Ong, A. Gupta, and P. Wei, "Minimalistic attacks: How little it takes to fool deep reinforcement learning policies," *IEEE Trans. Cogn. Devlop. Syst.*, vol. 13, no. 4, pp. 806–817, Dec. 2021.

[19] A. Pattanaik, Z. Tang, S. Liu, G. Bommannan, and G. Chowdhary, "Robust deep reinforcement learning with adversarial attacks," in *Proc. 17th Int. Conf. Auton. Agents MultiAgent Syst.*, 2018, pp. 2040–2042.

[20] K. L. Tan, Y. Esfandiari, X. Y. Lee, Aakanksha, and S. Sarkar, "Robustifying reinforcement learning agents via action space adversarial training," in *2020 IEEE Amer. Control Conf.*, 2020, pp. 3959–3964 .

[21] L. Schott, H. Hajri, and S. Lamprier, "Improving robustness of deep reinforcement learning agents: Environment attack based on the critic network," in *2022 IEEE Int. Joint Conf. Neural Netw.*, 2022, pp. 1–8.

[22] T. Chen, J. Liu, Y. Xiang, W. Niu, E. Tong, and Z. Han, "Adversarial attack and defense in reinforcement learning-from ai security view," *Cybersecurity*, vol. 2, pp. 1–22, 2019.

[23] J. Moos, K. Hansel, H. Abdulsamad, S. Stark, D. Clever, and J. Peters, "Robust reinforcement learning: A review of foundations and recent advances," *Mach. Learn. Knowl. Extraction*, vol. 4, no. 1, pp. 276–315, 2022.

[24] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.

[25] N. Rudin, D. Hoeller, P. Reist, and M. Hutter, "Learning to walk in minutes using massively parallel deep reinforcement learning," in *Proc. 5th Conf. Robot Learn.*, Nov. 2022, vol. 164, pp. 91–100.

[26] F. Grimminger et al., "An open torque-controlled modular robot architecture for legged locomotion research," *IEEE Robot. Automat. Lett.*, vol. 5, no. 2, pp. 3650–3657, Apr. 2020.

[27] W. Koch, R. Mancuso, R. West, and A. Bestavros, "Reinforcement learning for UAV attitude control," *ACM Trans. Cyber- Phys. Syst.*, vol. 3, no. 2, pp. 1–21, 2019.

[28] F. Fei, Z. Tu, D. Xu, and X. Deng, "Learn-to-recover retrofitting UAVs with reinforcement learning-assisted flight control under cyber-physical attacks," in *2020 IEEE Int. Conf. Robot. Automat.*, 2020, pp. 7358–7364.

[29] Leutenegger et al., "Flying robots," in *Springer Handbook of Robotics*. Berlin, Germany: Springer, 2016, pp. 623–670.

Authorized licensed use limited to: The University of Hong Kong Libraries. Downloaded on August 16,2026 at 15:50:17 UTC from IEEE Xplore. Restrictions apply.